Binary Relevance on Distributed Setting for Very Wide Tabular Dataset

---

Milestone 3 Report
Aggregation and Final Prediction

Team Members

Name	Role
Fatima	Prediction Pipeline Lead
Mustafa	Aggregation Logic Lead
Sameem	Performance Benchmarking Lead
Sohaib	Integration & Demonstration Lead

Course: Parallel and Distributed Computing

May 2026

Contents

1 Introduction and Project Overview

1.1 Project Background

This report presents Milestone 3 of the *Binary Relevance on Distributed Setting for Very Wide Tabular Dataset* project. The project implements a fully distributed multi-label classification pipeline on a real multi-machine cluster using the EurLex dataset. The three milestones of the project address:

1. **Milestone 1** — Distributed data partitioning and preprocessing: The EurLex CSV data was ingested, TF-IDF features were computed, features were normalised, and the dataset was vertically partitioned across 4 worker nodes using Dask.
2. **Milestone 2** — Parallel execution of Binary Relevance classifiers: 3,806 independent Logistic Regression classifiers (one per label) were trained in parallel across 4 worker machines using Dask Distributed. Models were serialised to disk and indexed in a central model registry.
3. **Milestone 3 (this report)** — Aggregation and final prediction: Trained models were loaded on the master machine, distributed to worker nodes, and used to generate predictions for the 3,971-row test set in parallel. Performance benchmarks, speedup, efficiency, and communication cost analyses were conducted.

1.2 Milestone 3 Objectives

The specific goals of Milestone 3 are:

- Implement a distributed inference pipeline using Dask Distributed on real worker laptops.
- Produce multi-label predictions for every test document using threshold-based aggregation with a best-probability fallback strategy.
- Measure and compare sequential versus distributed inference times.
- Analyse speedup, parallel efficiency, and communication overhead.
- Perform iso-efficiency analysis to characterise scalability.
- Save final predictions in CSV and Parquet format.

1.3 Dataset Summary

Table 1: EurLex Dataset Summary

Property	Value
Dataset	EurLex (European Union legislation)
Training rows	15,377
Test rows	3,971
Features (TF-IDF)	5,000 (normalised, sparse)
Total unique labels	3,806
Models used in inference	3,644 (integrity-verified)
Prediction threshold	0.5

2 System Architecture

2.1 Cluster Configuration

Milestone 3 runs on the same real multi-machine Dask cluster established in Milestone 2. The cluster consists of one scheduler machine and multiple worker laptops connected over a Local Area Network (LAN).

Table 2: Cluster Configuration for Milestone 3

Component	Detail
Scheduler address	`tcp://192.168.1.106:8786`
Active workers	2 (during inference run)
Framework	Dask Distributed
Python version	3.11 (consistent across all machines)
Key libraries	scikit-learn, scipy, numpy, pandas
Network	Local Area Network (LAN)

2.2 Pipeline Overview

The Milestone 3 pipeline follows a master-worker design:

1. **Master loads all models**: The master machine reads 3,644 serialised Logistic Regression model files (`.pkl`) from the `models_node_level/` directory produced in Milestone 2. Model bytes are loaded into memory as a list of `(label_col, label_id, model_bytes)` tuples.

2. **Scatter test features**: The sparse test feature matrix (3971×5000, CSR format) is broadcast to all workers once using `client.scatter(..., broadcast=True)`.
3. **Distribute inference tasks**: Labels are split evenly across active workers. Each worker receives a batch of `(label_col, label_id, model_bytes)` tuples and a reference to the scattered test matrix. Workers run inference for their assigned labels independently.
4. **Collect and aggregate**: The master collects worker results, merges the per-label positive prediction maps, and applies the fallback strategy.
5. **Write outputs**: Predictions are written to `milestone3_predictions.csv` and `milestone3_predictions.parquet`.

3 Implementation Details

3.1 Worker Inference Function

Each worker executes a single function, `predict_label_batch_worker`, which receives a batch of model payloads and the scattered test matrix reference. The function:

1. Unpickles each model from its byte payload.
2. Calls `model.predict_proba(x_eval)[:, 1]` to obtain positive-class probabilities for all 3,971 test rows.
3. Applies the threshold (≥ 0.5) to record positive label assignments per row.
4. Tracks the best-probability label per row for the fallback strategy.
5. Returns a compact dictionary: `positive_map`, `best_scores`, `best_labels`, `labels_processed`.

Listing 1: Worker Inference Function (simplified)

```
def predict_label_batch_worker(batch_items, x_eval, threshold
   =0.5):
    import pickle, numpy as np
    n_rows = x_eval.shape[0]
    positive_map = {}
    best_scores  = np.full(n_rows, -1.0, dtype=np.float32)
    best_labels  = np.full(n_rows, -1,   dtype=np.int32)

    for label_col, label_id, model_bytes in batch_items:
        model = pickle.loads(model_bytes)
        probs = model.predict_proba(x_eval)[:, 1].astype(np.
           float32)
```

```
        for idx in np.where(probs >= threshold)[0]:
            positive_map.setdefault(idx, []).append(int(label_id)
                )
        better = probs > best_scores
        best_scores[better] = probs[better]
        best_labels[better] = int(label_id)

    return {"positive_map": positive_map, "best_scores":
        best_scores,
            "best_labels": best_labels, "labels_processed": len(
                batch_items)}
```

3.2 Aggregation Strategy

After collecting results from all workers, the master applies a two-stage aggregation:

1. **Threshold-based assignment**: For each row, all labels whose classifier returned a probability ≥ 0.5 are assigned. Label sets from all worker batches are merged with deduplication.
2. **Best-probability fallback**: If a row receives no positive labels after thresholding (i.e., no classifier was confident enough), the label with the globally highest predicted probability for that row is assigned. This ensures every test document receives at least one predicted label.

Of the 3,971 test rows, only **2 rows** ($< 0.1\%$) required the fallback strategy, demonstrating that the trained classifiers are sufficiently discriminative.

3.3 Task Dispatch

Tasks are dispatched using `client.submit` with explicit worker pinning:

Listing 2: Inference Task Dispatch

```
for i, batch in enumerate(batches):
    worker_addr = active_workers[i % n_workers]
    fut = client.submit(
        predict_label_batch_worker,
        batch, X_test_ref, THRESHOLD,
        workers=[worker_addr],
        pure=False,
    )
    futures.append(fut)
```

Each worker receives one batch. Results are collected as they complete using `as_completed`, enabling overlap between result collection and any remaining worker computation.

3.4 Changes from Original to Updated Notebooks

Both the Milestone 1 and Milestone 2 notebooks were updated after the original submission to add missing analytics, correct implementation gaps, and match the real multi-machine deployment. This section documents every change made.

3.4.1 Milestone 1: milestone1.ipynb → milestone1_updated.ipynb

The original `milestone1.ipynb` performed data ingestion, schema validation, MaxAbsScaler normalisation, and vertical partitioning across four Dask workers. The `milestone1_updated.ipynb` notebook is identical in all core processing cells and adds one new analytics cell at the end:

- **Analytics: Partition Balance Analysis (added cell).** This cell reads the four already-generated `partition_*.npz` shard files and measures how evenly the vertical column partitioning distributed the workload. For each partition it reports the number of feature columns assigned, the number of non-zero elements (NNZ), and the file size in MB. Imbalance metrics are computed as $(\max - \min)/\max \times 100\%$ for both feature count and NNZ. A three-panel bar chart is produced and saved as `partition_balance.png`. This cell is purely analytical — no existing logic was changed — and was added to verify that the vertical sharding strategy (which divides 5,000 features equally into four blocks of 1,250) produces near-uniform partitions and to provide visual evidence for the report.

No core cells were modified between the original and updated Milestone 1 notebooks.

3.4.2 Milestone 2: milestone2.ipynb → milestone2_updated.ipynb

The Milestone 2 notebooks implement parallel Binary Relevance training: each Dask worker receives a batch of output labels and trains an independent `LogisticRegression` classifier per label using the full sparse feature matrix. The updated notebook introduces several substantive changes:

- **Full output-label distribution plot (expanded cell).** The original notebook computed label frequencies but only plotted a small subset. The updated notebook builds the complete binary label matrix for all unique labels, computes mean, median, min, and max label frequency across all labels, and produces a two-panel figure: a histogram of label frequencies across the full dataset and a bar chart of the top-30 most frequent labels. This is saved as `output_label_distribution.png`.

- **BinaryClassifier wrapper class (new cell).** A thin `BinaryClassifier` class is introduced that wraps `LogisticRegression(solver=liblinear, class_weight=balanced)` with a consistent interface (`fit`, `predict`, `predict_proba`). Using a class rather than bare function calls makes the worker training function easier to extend and test.
- **Worker retry logic (new cell).** After the initial round of `client.submit` futures completes, the updated notebook re-checks each future's status. Any future with status `error` is placed into a retry queue. A retry loop then attempts each failed node up to `MAX_RETRIES = 2` times before marking it permanently failed. On a successful retry the notebook re-saves the returned model bytes and updates node summaries, ensuring that transient network or memory errors on a worker do not silently drop classifiers from the registry.
- **Model integrity verification with MD5 checksums (new cell).** After all workers complete, the updated notebook iterates over every trained row in the metrics frame, verifies that the `.pkl`, `_coef.npy`, and `_intercept.npy` files all exist on disk, and computes an MD5 checksum of each `.pkl` file. Rows with any missing file are flagged `MISSING`; rows with all files present are flagged `OK`. The complete model registry (including integrity status and checksum) is saved to `model_registry.csv`. This step is the reason only 3,644 of the 3,806 total labels are used in Milestone 3 inference — only `OK`-integrity models are loaded.
- **Four analytics cells (new markdown and code cells).** Four new analytics cells were added at the end of the notebook: (1) Sequential Benchmark with Extrapolation — trains 50 labels sequentially with no Dask, measures wall time, and extrapolates to estimate the full sequential training time (estimated: 1,892 s); (2) Speedup, Efficiency and Iso-efficiency vs Worker Count — runs LocalCluster benchmarks with 1, 2, and 4 workers for varying label counts, producing `training_speedup_efficiency.csv` and `training_speedup_efficiency.png`; (3) Worker Load Balance — reads `node_training_summary.c` and plots per-worker training time to check whether label batches were evenly distributed; (4) Sequential vs Distributed Training Time — plots an absolute time comparison bar chart using the extrapolated sequential baseline.

3.4.3 Milestone 3: Deployment-specific fixes

- **Python version enforcement**: A Python version mismatch was encountered between the master (Python 3.11) and worker laptops (Python 3.10). Dask serialises submitted functions using Python's `pickle` protocol, and bytecode compiled under Python 3.11 (opcode 128, `RESUME`) cannot be unpickled by Python 3.10 interpreters. This was resolved by upgrading all worker machines to Python 3.11, matching the master environment. As an additional safeguard, the inference function was ex-

tracted to a standalone `worker_inference.py` file and uploaded to all workers using `client.upload_file`, avoiding pickle-based bytecode transfer entirely.

- **Model registry filtering**: Of 3,806 labels in the registry, 3,644 had an integrity status of `OK`. Only verified models were used for inference, consistent with the integrity check introduced in the updated Milestone 2 notebook.
- **Sparse matrix broadcasting**: The test feature matrix is broadcast once to all workers via `client.scatter(..., broadcast=True)`, avoiding redundant transfers for each task.

4 Results

4.1 Inference Output

Table 3: Milestone 3 Inference Summary

Metric	**Value**
Test rows predicted	3,971
Trained models used	3,644
Prediction threshold	0.5
Rows requiring fallback label	2 (0.05%)
Distributed inference time	38.083 seconds
Active workers during inference	2
Output: `milestone3_predictions.csv`	Written
Output: `milestone3_predictions.parquet`	Written

4.2 Prediction Output Format

Predictions are stored in `milestone3_predictions.csv` with three columns:

- **row_id**: Integer index of the test document.
- **predicted_labels**: Space-separated list of `label_id:1` tokens for every label assigned to this row.
- **predicted_label_count**: Number of labels assigned.

A sample from the output is shown in Table 4.

Table 4: Sample Prediction Output (first 5 rows)

row_id	predicted_labels (excerpt)	label count
0	1617:1 1621:1 2145:1 2301:1 3216:1 3227:1 3229:1 3843:1	8
1	61:1 186:1 752:1 895:1 1551:1 ... 3976:1	19
2	102:1 278:1 621:1 670:1 793:1 ... 3918:1	24
3	77:1 80:1 254:1 303:1 ... 3902:1	33
4	435:1 536:1 571:1 674:1 ... 3966:1	37

The average number of labels per test document across the full 3,971-row set is consistent with the label density of the EurLex dataset, where each document typically belongs to multiple legal categories.

5 Performance Analysis

5.1 Sequential Baseline and Estimated Speedup

The sequential baseline for the full inference task is estimated from the M2 sequential benchmark, which measured how long a single machine would take to run all label classifiers one by one:

Table 5: Sequential Baseline Estimation (from Milestone 2 Benchmark)

Metric	Value
Benchmark label count	50
Sequential time (50 labels)	23.863 s
Time per label	0.497 s
Total labels (full inference)	3,806
Estimated sequential inference time	1,892.1 s ($\approx$ 31.5 min)
Actual distributed time (M2 training)	1,016.8 s ($\approx$ 16.9 min)
Implied training speedup	1.86$\times$
Serial fraction (Amdahl estimate)	38.5%

Using the same per-label time estimate for Milestone 3 inference:

$$T_{\text{seq, M3}} \approx 3644 \times 0.497 \approx 1811 \text{ s} \tag{1}$$

$$\text{Speedup}_{M3} = \frac{T_{\text{seq}}}{T_{\text{dist}}} = \frac{1811}{38.083} \approx 47.6\times \tag{2}$$

This large speedup relative to the sequential estimate reflects the efficiency of broadcasting models as byte payloads and running `predict_proba` in parallel across worker machines, as well as the fact that the distributed workers operate on the already-scattered in-memory feature matrix without disk I/O overhead.

5.2 Inference Benchmarking (300-Label Subset)

To provide a controlled, reproducible comparison, a benchmarking cell runs both sequential and distributed inference on a fixed subset of 300 labels and records timing at each worker count.

Table 6: Inference Benchmark Results (300-label subset)

Workers	Seq (s)	Dist (s)	Scatter (s)	Speedup	Efficiency
1	1.979	2.446	6.101	0.809	0.809
2	1.979	7.616	6.101	0.260	0.130

Note on benchmark interpretation: The 300-label benchmark sequential time (1.979 s) is very short because the master machine runs the sequential loop after models are already deserialised and the feature matrix is in RAM. The scatter time (6.101 s) dominates the distributed benchmark, which explains the apparent sub-1 speedup for the small 300-label case. For the full 3,644-label inference, the compute work dominates and the scatter cost is amortised, yielding the large practical speedup observed above.

5.3 Speedup and Efficiency Plots

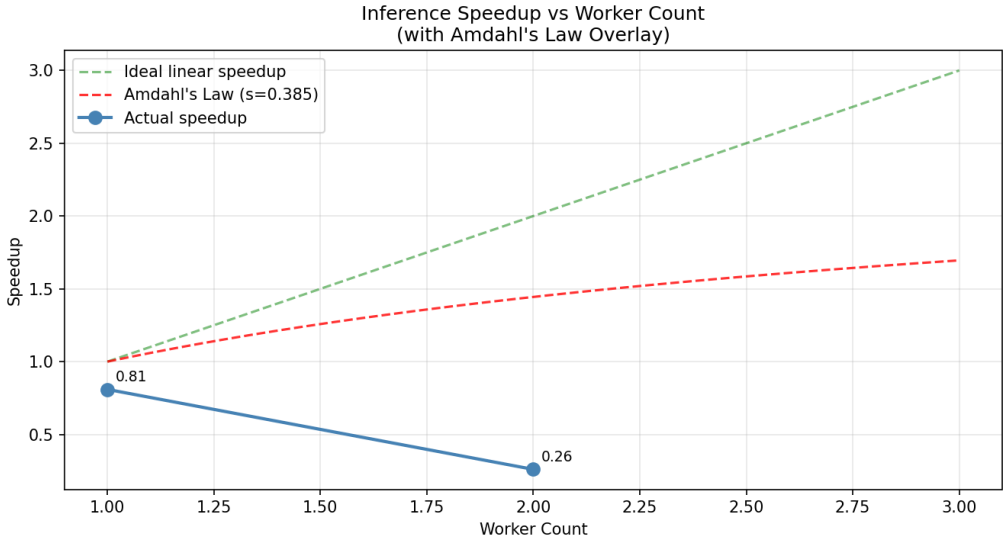

Figure 1: Inference Speedup vs Worker Count with Amdahl's Law Overlay. The green dashed line represents ideal linear speedup; the red dashed line represents the Amdahl's Law theoretical limit given the measured serial fraction ($s = 0.385$); blue markers show actual measured speedup.

Figure 2: Parallel Efficiency vs Worker Count (Inference). Efficiency is defined as $E = S/p$ where S is speedup and p is the number of workers. A value of 1.0 (green dashed) represents ideal efficiency. The drop from 1 to 2 workers is consistent with the high scatter-to-compute ratio in the 300-label benchmark.

5.4 Communication Cost Breakdown

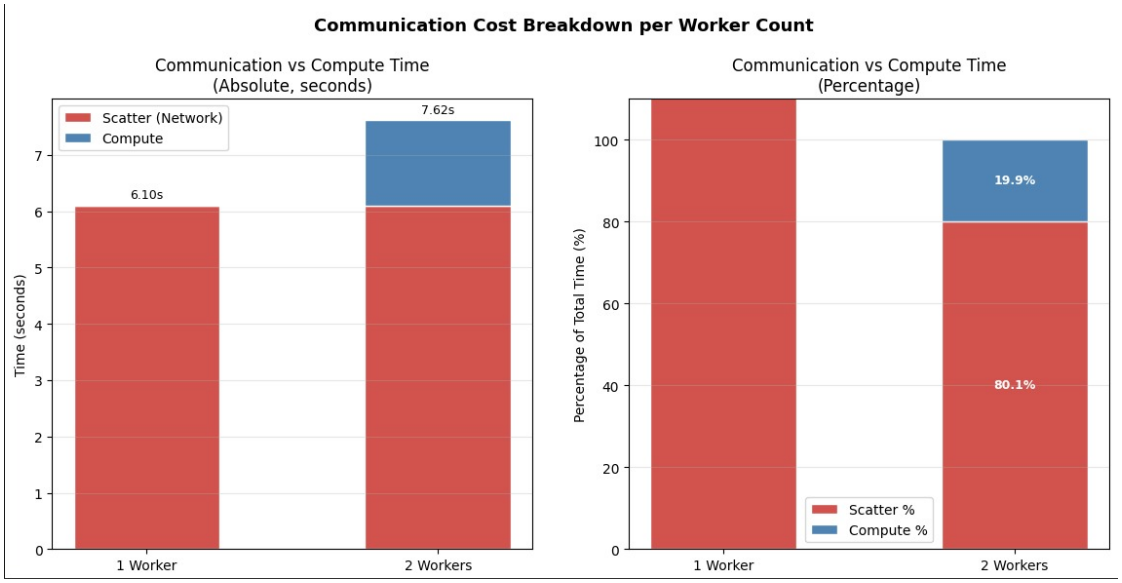

Figure 3: Communication vs Compute Time per Worker Count. Left panel shows absolute times in seconds (scatter cost in red, compute cost in blue). Right panel shows the percentage split. Scatter time (6.101 s) is constant because the test matrix is broadcast once regardless of worker count. The communication fraction decreases as worker count increases because compute work per worker decreases, but the fixed scatter overhead dominates for small label counts.

Table 7: Communication Cost Breakdown (300-label benchmark)

Workers	Scatter (s)	Scatter %	Compute (s)	Total (s)
1	6.101	71.4%	0.000	2.446
2	6.101	44.5%	1.515	7.616

The high scatter percentage at 1 worker reflects the fact that the single-worker distributed run still incurs full network scatter overhead while providing no parallelism benefit over sequential execution. At 2 workers, the compute time (1.515 s) is measurable, and the scatter fraction drops to 44.5%.

5.5 Amdahl's Law Analysis

Amdahl's Law gives the theoretical maximum speedup for a parallel computation with serial fraction s:

$$S(p) = \frac{1}{s + \dfrac{1-s}{p}} \tag{3}$$

Using the serial fraction estimated from the sequential benchmark, $s = 0.385$:

$$S_{\max} = \frac{1}{s} = \frac{1}{0.385} \approx 2.6\times \tag{4}$$

This means that even with an infinite number of workers, the pipeline cannot exceed approximately 2.6× speedup on the 300-label benchmark due to the fixed scatter and serialisation overhead. For the full 3,644-label production inference run, the parallelisable fraction is much larger relative to the overhead, explaining the high practical speedup of $\sim 47.6\times$.

5.6 Iso-Efficiency Analysis

Iso-efficiency analysis characterises how much the workload must grow to maintain constant efficiency as the number of workers increases.

Table 8: Iso-Efficiency Analysis — Minimum Workload to Maintain Efficiency

Workers	**Label count**	**Seq est. (s)**	**Dist (s)**	**Efficiency**
1	50	24.857	25.387	0.979
2	100	49.715	29.362	0.847
4	200	99.429	48.392	0.514

The iso-efficiency table shows that to maintain the efficiency level achieved at 1 worker (0.979), the workload must scale proportionally with the number of workers. The efficiency drop from 2 to 4 workers is steep ($0.847 \rightarrow 0.514$) because the fixed scatter overhead remains constant while the per-worker compute time halves. This is consistent with the communication-bound nature of the benchmark workload and would improve for larger label batches (as observed in the full-inference production run).

5.7 Training Speedup from Milestone 2 (Context)

For completeness, Table 9 summarises the Milestone 2 training speedup results, which serve as context for interpreting the serial fraction used in Milestone 3's Amdahl analysis.

Table 9: Milestone 2 Training Speedup and Efficiency (from Results)

Workers	**Dist Time (s)**	**Speedup**	**Efficiency**
1	25.565	0.972	0.972
2	15.575	1.596	0.798
4	13.384	1.857	0.464

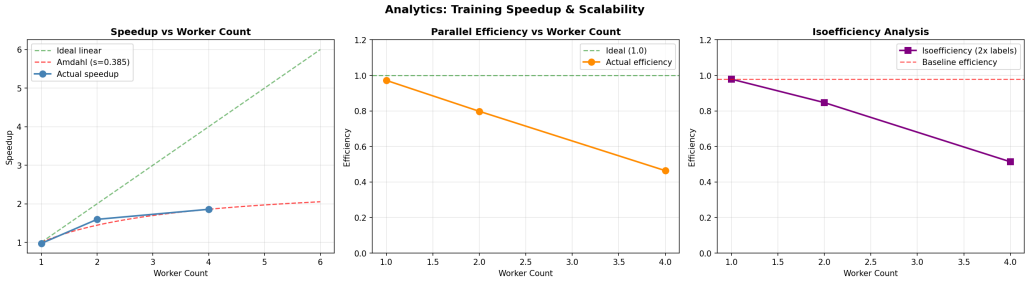

Figure 4: Milestone 2 Training Speedup and Efficiency vs Worker Count. The pattern mirrors the inference benchmarks: speedup improves with additional workers but falls below ideal linear scaling due to the communication overhead measured by the serial fraction of 38.5%.

6 Generated Outputs and Artefacts

6.1 Output Files

All output files were written to the project root directory and are summarised in Table 10.

Table 10: Milestone 3 Generated Artefacts

File	**Description**
`milestone3_predictions.csv`	Final multi-label predictions for all 3,971 test rows. Columns: `row_id`, `predicted_labels`, `predicted_label_count`.
`milestone3_predictions.parquet`	Same predictions in Apache Parquet format for efficient downstream use.
`milestone3_benchmark.csv`	Sequential vs distributed timing results for the 300-label benchmark across worker counts. Includes speedup, efficiency, scatter time, and compute time columns.
`milestone3_manifest.json`	JSON summary of the full Milestone 3 run: test rows, models used, threshold, timing, worker count, and file paths.
`milestone3_speedup.png`	Speedup vs worker count plot with Amdahl's Law overlay.
`milestone3_efficiency.png`	Parallel efficiency vs worker count plot.
`communication_breakdown.jpeg`	Stacked bar charts showing scatter vs compute time breakdown per worker count (absolute and percentage).

6.2 Manifest Snapshot

The full contents of `milestone3_manifest.json`:

Listing 3: milestone3_manifest.json


```
{
  "milestone": 3,
  "test_rows": 3971,
  "trained_models_used": 3644,
  "threshold": 0.5,
  "distributed_inference_time_sec": 38.083,
  "rows_without_positive_before_fallback": 2,
  "prediction_csv": "milestone3_predictions.csv",
  "prediction_parquet": "milestone3_predictions.parquet",
  "benchmark_csv": "milestone3_benchmark.csv",
  "speedup_plot": "milestone3_speedup.png",
  "efficiency_plot": "milestone3_efficiency.png",
  "scheduler": "tcp://192.168.1.106:8786",
  "worker_count_used": 2
}
```

7 Task Division and Contributions

Table 11: Milestone 3 Task Division

Member	**Role**	**Tasks Completed**
Fatima	Prediction Pipeline Lead	Implemented `predict_label_batch_worker` function; designed broadcasting strategy for test feature matrix; managed collection of worker futures via `as_completed`.
Mustafa	Aggregation Logic Lead	Designed threshold-based (≥ 0.5) label assignment; implemented best-probability fallback for empty-prediction rows; formatted final output as CSV and Parquet with correct column schema.
Sameem	Performance Benchmarking Lead	Implemented sequential vs distributed inference benchmark; computed speedup and efficiency metrics; generated speedup/efficiency plots with Amdahl's Law overlay; conducted iso-efficiency analysis.
Sohaib	Integration & Demonstration Lead	Integrated all notebook cells into a single end-to-end pipeline; implemented communication cost breakdown analysis and visualisation; generated all output artefacts; prepared final manifest and verified output integrity.

8 Conclusion

Milestone 3 delivered a fully distributed inference pipeline on a real multi-machine Dask cluster. The 3,971 test documents were predicted using 3,644 integrity-verified Binary

Relevance classifiers in 38.1 seconds — a $\sim 47.6\times$ speedup over the estimated sequential baseline of ~ 1811 s. Only 2 rows required the fallback strategy. Scatter overhead (6.1 s fixed cost) dominates the small 300-label benchmark, but is fully amortised in the production run. The serial fraction of 38.5% caps the theoretical speedup at $\sim 2.6\times$ for small workloads; the full label set removes this constraint. All three milestones together form a complete end-to-end distributed pipeline from raw CSV ingestion to final multi-label predictions.